Far lampeggiare un LED: setup, loop e delay

Lezione 6 — 1MAK, SST-INF-01c — Dal circuito al primo programma che lampeggia davvero

16/10/2026

1 Da dove ripartiamo: Arduino e i suoi pin

Arduino è una **scheda elettronica**, una scatolina dotata di **terminali** (i **pin**), disponibili su dei **connettori** in cui si infilano i cavetti. Questi terminali si possono **programmare** con un codice: possono **fornire** o **non fornire** corrente.

Definizione

Sul lato dei pin digitali ci sono **14 terminali** (da 0 a 13) che possiamo decidere di **accendere** o **spegnere**. I primi due, lo **0** e l'**1**, è meglio **non usarli**, per motivi che vedremo con il tempo (servono alla comunicazione con il computer). Quindi:

- alla domanda "quanti terminali digitali ha Arduino?" la risposta è **14**;
- alla domanda "quanti ne possiamo usare senza problemi?" la risposta è **12**.

Attenzione

Il terminale da collegare **ogni volta** è **GND**: è il riferimento a **0 volt**, cioè il **terminale negativo** del generatore. Ogni circuito che parte da un pin deve **tornare a GND**, altrimenti il circuito non è chiuso e la corrente non passa.

2 Un pin è come un generatore che si accende e si spegne

Prendiamo il **pin 7**: colleghiamo un filo, mettiamo un **resistore** e **chiudiamo il circuito** su **GND**. È lo stesso circuito visto nella parte di elettronica: un **generatore** e un **resistore**.

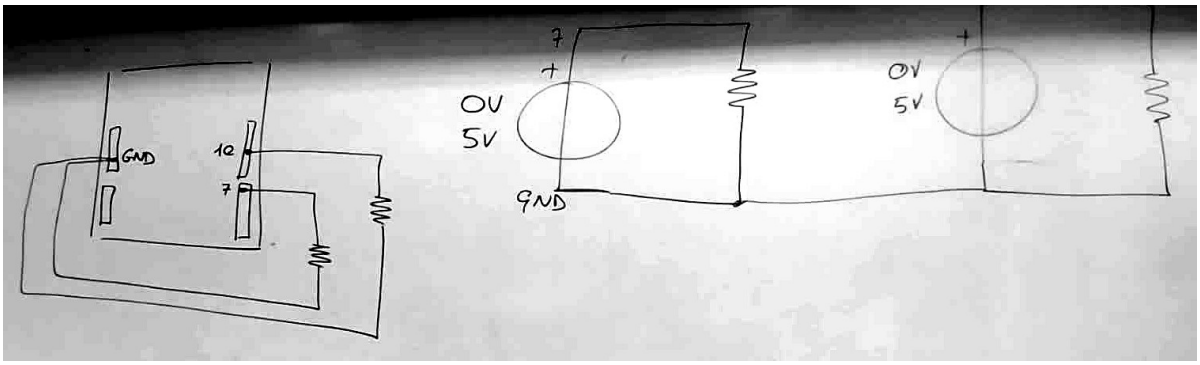


Figura 1: La lavagna. A sinistra Arduino come una scatola: dal pin 7 e dal pin 12 partono due circuiti, ciascuno con il suo resistore, che tornano entrambi a GND. A destra lo schema equivalente: ogni pin è un generatore che può dare 0 V o 5 V, e i due generatori hanno il GND in comune.

Definizione

Con il programma possiamo dire a questo "generatore" di produrre **0 volt** oppure **5 volt**:

- con **0 V** non c'è nessuna **forza** che spinge gli elettroni: **non passa corrente**, e un'eventuale lampadina collegata resta **spenta**;
- con **5 V** c'è una **forza** che spinge gli elettroni: gli elettroni cominciano a **scorrere**, passa **corrente**, e la lampadina si **accende**.

Quindi da programma possiamo dire: **fai passare la corrente** oppure **non farla passare**.

Con 12 pin utilizzabili si possono realizzare **tanti circuiti** quanti ne servono. Per esempio sul **pin 12** si fa un altro circuito, con un altro resistore, che torna anche lui su **GND**: è come avere un **secondo generatore**, con il GND **in comune** con il primo. Anche questo può andare da 0 a 5 V, e possiamo decidere quale circuito accendere **in base alle nostre esigenze**.

Esempio

Il resistore sta per "qualunque cosa". Nello schema il resistore rappresenta un **qualunque dispositivo** da comandare:

- un vero **resistore**, per esempio quello di una stufetta che **scalda**;
- una **lampadina**, per un sistema di accensione delle luci di una casa;
- un **motore**, una **pompa** che porta l'acqua in un impianto, una **ventola** che raffredda un dispositivo troppo caldo (qui con una certa licenza: per comandarli serve un circuito un po' più complesso);
- un **LED**, una piccola lampadina che possiamo far **accendere** e **lampeggiare** in base a quello che vogliamo comunicare.

Nota

I vecchi **display a scritte** delle fermate degli autobus (oggi spesso sostituiti da monitor) erano fatti di **tante lampadine**, cioè tanti LED: accendendoli nel modo giusto si **forma la scritta**. A decidere quali LED accendere, come, dove e in che momento, c'è un microcontrollore come quello di Arduino. Con Arduino si può realizzare davvero di tutto: dipende dalla **fantasia** e dall'**esercizio**.

3 Prima la teoria, poi la pratica

Nota

Il nostro corso punta molto sulla **pratica**: l'importante è **saper fare** le cose. Ma senza **teoria** la pratica è faticosa. È come mettere una scimmietta davanti a una macchina da scrivere: anche se batte i tasti per ore, non diventa un poeta, se nessuno le spiega come si scrive un testo.

4 Le parole del primo programma

Alla lavagna sono state raccolte le **parole** viste finora, che servono per scrivere il primo programma. Si trovano tutte nelle dispense precedenti, in particolare nella A05:

Parola	A che cosa serve
`void`	parola che precede il nome di un blocco di codice
`setup`	blocco eseguito una volta sola, all'inizio
`loop`	blocco ripetuto all'infinito
`pinMode`	decide se un pin è un ingresso o un'uscita
`INPUT` / `OUTPUT`	ingresso (orecchio) / uscita (bocca)
`digitalWrite`	accende o spegne un pin di uscita
`HIGH` / `LOW`	acceso (5 V) / spento (0 V)
`delay`	ferma la macchina per un certo tempo

Attenzione

Le dispense vanno studiate. Queste parole erano già nella dispensa pubblicata dopo la lezione precedente: a lezione si parte dal presupposto che le dispense siano state **lette**, possibilmente più volte. Per **martedì** questi comandi vanno saputi.

5 Il blocco setup

Definizione

Ogni programma Arduino ha delle **zone ben definite**. La prima è il blocco `void setup()`:

- ce n'è **uno solo**;
- viene eseguito **una volta sola**, ogni volta che si accende la macchina o le si carica dentro il programma;
- è racchiuso tra due **graffe**: quella aperta `{` segna l'inizio, quella chiusa `}` la fine del blocco;
- dentro si mettono **solo le impostazioni iniziali** della macchina, per esempio la direzione dei pin.

5.1 pinMode: una bocca sul pin 10

Esempio

La bocca. Se vuoi dire qualcosa alla classe, che cosa usi? Il naso? Gli occhi? No: la **bocca**. La bocca è una "periferica" del corpo che serve a **emettere** suoni e comunicare qualcosa agli altri: dal corpo **esce** qualcosa. Per ascoltare, invece, si usano le **orecchie**: qualcosa **entra**.

Vogliamo creare una "bocca" sul **pin 10**, in modo che Arduino possa dire qualcosa verso l'esterno. Dobbiamo dire ad Arduino che il pin 10 è un'**uscita**:

```
pinMode(10, OUTPUT);
```

Tra le parentesi tonde: il **numero del pin** (10, ma può essere 3, 7, 13, quello che ci serve), una **virgola**, e `OUTPUT`. Se invece volessimo un "orecchio", cioè un pin che **ascolta** cosa hanno da dire altri dispositivi esterni, per esempio sul pin 5, scriveremmo:

```
pinMode(5, INPUT);
```

Nota

Per il momento useremo solo le **uscite** (la bocca): per usare bene gli ingressi (le orecchie) bisogna imparare cose nuove.

5.2 Le regole di scrittura

Attenzione

- Ogni istruzione del C termina con il **punto e virgola** `;`. Sempre. Se lo dimenticate, la macchina segnala un **errore**.
- Il C è sensibile alle **maiuscole e minuscole**: i comandi vanno scritti **esattamente** così. `pinMode` ha la **M maiuscola**; `INPUT` e `OUTPUT` sono **tutto maiuscolo**. Scrivere `pinmode` genera un errore.
- "Ho sbagliato solo una maiuscola" non basta: il programma è **sbagliato**. La programmazione **non lascia spazio** all'**errore sintattico** (cioè di scrittura): la macchina non dice "va bene, ho capito lo stesso".

6 Il blocco loop e digitalWrite

Definizione

Dopo il setup, la macchina inizia a **ripetere all'infinito** un altro blocco: **`void loop()`**. Tra le sue graffe si scrive il codice da **ripetere continuamente**. Se si lascia accesa la macchina per mille anni, lei lo ripete per mille anni: non si stufa, non si stanca, e finché ha energia continua.

Definizione

Per **accendere** o **spegnere** un circuito, cioè un pin, si usa **`digitalWrite`** (con la **W maiuscola**). Tra le tonde: il **pin**, una virgola, e **`HIGH`** (acceso) oppure **`LOW`** (spento). E sempre il punto e virgola.

```
digitalWrite(10, HIGH);   // accende il circuito del pin 10
digitalWrite(10, LOW);    // lo spegne
```

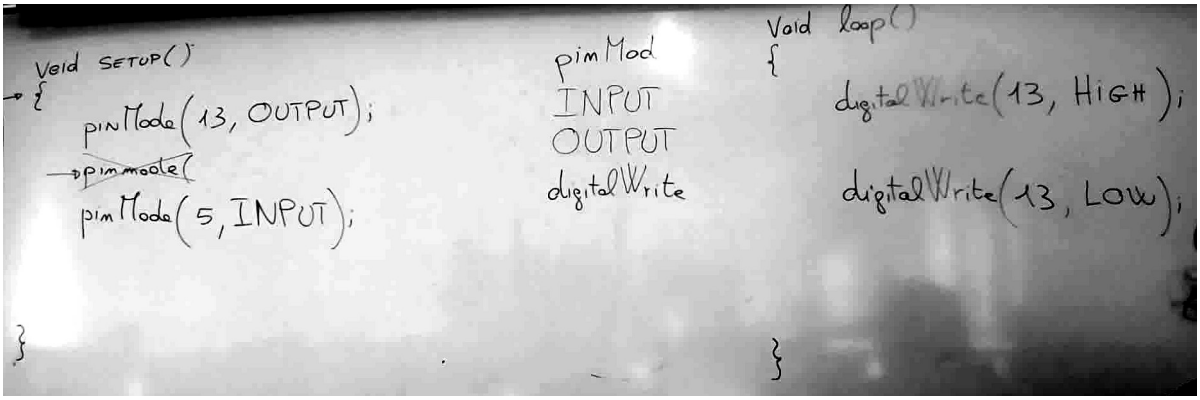


Figura 2: Il codice alla lavagna: a sinistra il setup con `pinMode(13, OUTPUT);` e `pinMode(5, INPUT);` (la scritta `pimmode(` è barrata: scritta così è un errore); al centro le parole da scrivere con le maiuscole giuste; a destra il loop con `digitalWrite(13, HIGH);` e `digitalWrite(13, LOW);`.

7 Proviamo in Tinkercad

Nel simulatore al pin 10 non è collegato niente. Per non complicarci la vita usiamo il **pin 13**, che ha già una **lampadina** (il LED **L**) collegata sulla scheda.

7.1 Primo tentativo: solo HIGH

```
void setup()
{
  pinMode(13, OUTPUT);
}

void loop()
{
  digitalWrite(13, HIGH);
}
```

Nota

Conviene scrivere subito **tutte e due le graffe**, aperta e chiusa, prima di riempire il blocco: se se ne dimentica una, il programma dà **errore** e non funziona niente.

Avviando la simulazione la lampadina si **accende**. Il programma sta dicendo continuamente "accendi il LED, accendi il LED, accendi il LED…": la macchina non si stufa e non risponde "me l'hai già detto".

7.2 Secondo tentativo: HIGH e poi LOW

```
void loop()
{
  digitalWrite(13, HIGH);
  digitalWrite(13, LOW);
}
```

Il LED è acceso o spento? A vederlo sembra **sempre acceso** (o quasi). In realtà sta **lampeggiando**, ma **troppo velocemente** perché l'occhio se ne accorga. Con uno **strumento di misura** si vedrebbe che si accende e si spegne continuamente.

Definizione

Arduino ha un **clock** interno da **16 MHz**: esegue circa **16 milioni** di operazioni **al secondo**. Accendere e spegnere un pin non è una sola operazione, ne servono di più, ma il LED si accende e si spegne comunque **centinaia di migliaia** di volte al secondo: mentre noi diciamo "uno", lui si è già acceso e spento moltissime volte.

Nota

Precisazione rispetto alla lezione: in aula è stata data, a grandi linee, una stima di circa 500.000 lampeggi al secondo. Con i comandi di Arduino (`digitalWrite` richiede qualche microsecondo) il valore reale è più basso, nell'ordine di qualche centinaio di migliaia di cambi di stato al secondo; il concetto non cambia: è **troppo veloce** per l'occhio.

Esempio

Gli FPS dei videogiochi. Gli **FPS** (*frame per second*, fotogrammi al secondo) dicono quante volte al secondo il computer crea e manda al monitor un'**immagine nuova**. Per far vedere un personaggio che si muove, mostra una fotografia, poi un'altra con il piede un po' più avanti, poi un'altra ancora. A 120 FPS, in un secondo passano 120 fotografie, ognuna pochissimo diversa dalla precedente, e il movimento sembra **fluido**. Sopra i **25 fotogrammi al secondo** l'occhio non si accorge più delle singole immagini. Nei giochi più veloci servono molti più FPS, per non vedere scatti.
Un monitor 1920×1080 ha circa **2 milioni** di "lampadine" (i pixel) da accendere e spegnere con criterio; qui ne abbiamo **una sola**, e Arduino la accende e spegne velocissimo. Se invece gli chiedessimo di disegnare un'immagine da 2 milioni di pixel, andrebbe lentissimo: è una macchina veloce per operazioni **molto semplici**.

8 delay: diamo tempo all'occhio

Definizione

Per vedere il lampeggio bisogna dire alla macchina di **aspettare** tra l'accensione e lo spegnimento: lo si fa con **`delay`**. Il comando **blocca** Arduino (lo "freeza", lo fa "laggare" apposta) per il numero di **millisecondi** scritto tra le tonde. `delay(1000)` vuol dire mille millisecondi, cioè **un secondo**.

Attenzione

Il `delay` serve **due volte**: dopo l'`HIGH` e anche dopo il `LOW`. Con un solo `delay` dopo l'accensione, il LED starebbe acceso un secondo, poi si spegnerebbe e subito, senza attesa, il loop ricomincerebbe accendendolo di nuovo: lo spegnimento **non si vedrebbe**.

```
void loop()
{
  digitalWrite(13, HIGH);
  delay(1000);    // acceso per un secondo
  digitalWrite(13, LOW);
  delay(1000);    // spento per un secondo
}
```

Ora la lampadina si vede **lampeggiare**: accesa un secondo, spenta un secondo.

8.1 Più veloce, più veloce…

Valore del delay	Che cosa si vede nel simulatore
1000	lampeggio lento, un secondo acceso e un secondo spento
500	lampeggio più veloce
200	ancora più veloce
100	si vede ancora lampeggiare
60	si vede “qualcosa”, a scatti
50	quasi non si vede più: non fa in tempo a spegnersi che deve già riaccendersi

Nota

Il limite del simulatore. Tinkercad simula una macchina **su un’altra macchina**: è molto **più lento** di un Arduino vero, e non si può apprezzare il lampeggio come nella realtà. Con la scheda vera si riesce a vedere il lampeggio fino a valori di circa **10–15 millisecondi**. Quando lo faremo dal vivo la differenza si vedrà bene.

Esempio

Perché a volte sembra che cambi la luminosità. Con valori bassi il LED del simulatore può sembrare non più acceso e spento, ma **più o meno luminoso**. Non è così: è un’**interferenza** tra la frequenza con cui si aggiorna il **monitor** (i suoi FPS) e la frequenza del lampeggio. È lo stesso **effetto ottico** di quando, in macchina, si sorpassa un’altra auto e sembra che le sue **ruote girino al contrario**: il cervello “scatta foto” continuamente, ma se la ruota gira troppo veloce si perdono dei momenti, e nella sequenza la ruota sembra tornare indietro invece di andare avanti. Con la scheda vera, il lampeggio si vede in modo diverso.

9 Cosa ricordare di questa lezione

- Arduino ha **14 pin digitali**; lo **0** e l’**1** non si usano, quindi ne restano **12**. Ogni circuito parte da un pin e torna a **GND**.
- Un pin è come un **generatore** che il programma porta a **0 V** (spento) o a **5 V** (acceso); al posto del resistore può esserci qualunque dispositivo.
- **`void setup()`** viene eseguito una volta sola e contiene le impostazioni (es. `pinMode(13, OUTPUT);`); **`void loop()`** viene ripetuto all’infinito.

- Punto e virgola alla fine di ogni istruzione; maiuscole esatte: `pinMode`, `digitalWrite`, `INPUT`, `OUTPUT`, `HIGH`, `LOW`. Un errore di scrittura è un errore.
- Senza `delay` il LED lampeggia **troppo veloce** per l'occhio (clock da 16 MHz). Serve un **`delay`** dopo l'`HIGH` e uno dopo il `LOW`; il numero è in **millisecondi**.
- Il simulatore è più lento della scheda vera; con valori bassi può comparire un'**interferenza** con il monitor.